CSC_3TC34_TP Logique et fondements de l'informatique

AU 26-27 S1 — github/omgeta

1. Fonctions récursives

Idée: ce qu'un algorithme peut ou ne peut pas faire (ressources finies mais illimitées, efficacité ignorée).

i. **Church-Turing**: générales récursives = λ-calcul = machines de Turing (équivalences constructives)

ii. Codage de Gödel: tout est entier; $\langle m, n\rangle = m + \frac{1}{2}(m+n)(m+n+1)$, listes $\langle\langle a_0, \dots\rangle\rangle$

Partielles $f : \mathbb{N}^k \dashrightarrow \mathbb{N}$: $f(n)\downarrow$ défini (termine), $f(n)\uparrow$ non défini (boucle); $f(n)\downarrow = g(m)$: définis et égaux; **totale**: définie partout; un argument $\uparrow$ rend le tout $\uparrow$.

Primitives récursives (PR)

Plus petite classe contenant projections, constantes, successeur, stable par composition et **récursion primitive**:

$$f(x, 0) = g(x), \quad f(x, z+1) = h(x, f(x, z), z)$$

i. $=$ **boucles bornées** a priori: toujours totales

ii. $+$, $\times$, x^z, $\max(u-v, 0)$, $u\%v$, $\lfloor u/v\rfloor$, tests, $\langle m, n\rangle$: p.r.; Ackermann: calculable, **pas** p.r.

iii. Numérotation ψ_e: e = code (intention), ψ_e = fonction (extension)

iv. **Pas d'universalité**: $(e, x) \mapsto \psi_e(x)$ pas p.r. (diagonale $u(e, x) + 1$)

Générales récursives ($\mathcal{R}$)

PR + **opérateur** μ: $\mu g(x)$ = plus petit z avec $g(z, x) = 0$ et $g(i, x)\downarrow > 0$ pour $i < z$ (boucle `while` non bornée).

i. Fonctions **partielles**; numérotation $\varphi_e^{(k)}$, $\varphi_e := \varphi_e^{(1)}$; récursive = calculable

Théorèmes fondamentaux

Universalité: $\exists u,\ \varphi_u^{(2)}(e, x) = \varphi_e(x)$ (interpréteur).

Forme normale (Kleene): T prédicat p.r., U p.r.:

$$\varphi_e(x) = U\big(\mu n.\, T(n, e, x)\big)$$

$T(n, e, x)$: «n code un arbre de calcul de $\varphi_e(x)$» (vérification p.r.); $\varphi_e(x)\downarrow \iff \exists n. T(n, e, x)$.

i. **Lancement en parallèle**: $\mu n.(T(n, e_1, x) \vee T(n, e_2, x))$ termine ssi $\varphi_{e_1}(x)\downarrow$ ou $\varphi_{e_2}(x)\downarrow$

s-m-n: $s_{m,n}$ **p.r.** telle que

$$\varphi^{(n)}_{s_{m,n}(e, x_1..x_m)}(y_1..y_n) = \varphi_e^{(m+n)}(x_1..x_m, y_1..y_n)$$

Récursion de Kleene (astuce de Quine): $h : \mathbb{N}^{k+1} \dashrightarrow \mathbb{N}$ récursive $\Rightarrow \exists e$:

$$\varphi_e^{(k)}(x) = h(e, x)$$

Un programme peut utiliser son propre code. Preuve: $s = s_{1,k}$, $(t, x) \mapsto h(s(t, t), x) = \varphi_c(t, x)$, $e := s(c, c)$.

Point fixe (Kleene-Rogers): F calculable totale $\Rightarrow \exists e$, $\varphi_e = \varphi_{F(e)}$.

Exemple: prédiction impossible (CC 25-26)

Pas d'algorithme total f avec $f(e) = 0$ si $\varphi_e(0)\downarrow = 0$, 1 si $\varphi_e(0)\downarrow \neq 0$, arbitraire sinon.

i. Par Kleene, e: ignore son argument, calcule $f(e)$, renvoie 1 si $f(e) = 0$, sinon 0 (termine toujours)

ii. $f(e) = 0 \Rightarrow \varphi_e(0) = 1 \Rightarrow f(e) = 1$; $f(e) \neq 0 \Rightarrow \varphi_e(0) = 0 \Rightarrow f(e) = 0$: contradiction

iii. Conséquence: l'arbre de Kleene K (décidable, p.r., infini) n'a **aucune branche infinie calculable**, bien que Kőnig en donne une: Kőnig non constructif

Problème de l'arrêt

$H = \{(e, x) : \varphi_e(x)\downarrow\}$ **indécidable**.

Preuve (diagonale): si $h = \mathbf{1}_H$ calculable, $v(e) = 42$ si $h(e, e) = 0$, $\uparrow$ sinon; $v = \varphi_c$:

i. $\varphi_c(c)\downarrow \Rightarrow h(c, c) = 1 \Rightarrow v(c)\uparrow = \varphi_c(c)$: absurde

ii. $\varphi_c(c)\uparrow \Rightarrow h(c, c) = 0 \Rightarrow v(c)\downarrow$: absurde

i. H est semi-décidable (exécuter), $\complement H$ ne l'est pas

ii. Aussi indécidables: $\{e : \varphi_e(e)\downarrow\}$, $\{e : \varphi_e(0)\downarrow\}$ (via s-m-n)

Machines de Turing

États $Q = \{0..m-1\}$ (0 = arrêt), symboles $\{0, 1\}$, programme $\delta : (Q \setminus \{0\}) \times \Sigma \to Q \times \Sigma \times \{L, R\}$; configuration (état, bande finie, position).

i. Entrée $01^{x_1}01^{x_2}0\cdots$ (unaire), sortie 01^y0

ii. Configuration après n étapes: p.r.; MT $\equiv$ récursives

iii. Arrêt depuis bande vierge indécidable (réduction)

Méthode (rédiger un algorithme)

i. **p.r.**: n'utiliser que des boucles bornées (calculer la borne d'abord)

ii. **Calculable**: boucles non bornées, universalité («exécuter $\varphi_e(x)$»)

iii. Plusieurs recherches: **en parallèle** (entrelacer $n = 0, 1, 2, \dots$), jamais en séquence si l'une peut boucler

iv. Auto-référence: «par Kleene, e connaît son propre code»

2. Décidabilité et λ-calcul

i. $A \subseteq \mathbb{N}^k$ **décidable** (récursive): $\mathbf{1}_A$ calculable (répond oui/non)

ii. **Semi-décidable** (calculablement/récursivement énumérable): $n \mapsto 1$ si $n \in A$, $\uparrow$ sinon calculable (répond oui/...)

$$A \text{ décidable} \iff A \text{ et } \complement A \text{ semi-décidables}$$

($\Leftarrow$: lancement en parallèle des deux semi-algorithmes.)

Caractérisations du semi-décidable B:

i. $B = \{m : \varphi_e(m)\downarrow\}$ (domaine d'une calculable)

ii. $B = f(A)$, A décidable, f totale calculable

iii. $B = \emptyset$ ou $B = f(\mathbb{N})$, f totale calculable («énumérable»)

iv. $B = \{m : \exists n.\, R(n, m)\}$, R décidable (projection)

Stabilité	Décid.	Semi-déc.
$\cup$, $\cap$ finies	oui	oui (parallèle)
complémentaire	oui	**non**
projection $\exists$	**non**	oui
image calculable	non	oui

Théorème de Rice

$\mathcal{F} \subseteq \mathcal{R}^{(1)}$ ensemble de fonctions partielles calculables:

$$\{e : \varphi_e \in \mathcal{F}\} \text{ décidable} \Rightarrow \mathcal{F} = \emptyset \text{ ou } \mathcal{F} = \mathcal{R}^{(1)}$$

Aucune propriété non triviale de la **fonction** calculée (extension) n'est décidable sur le programme.

i. Indécidables: $\{e : \varphi_e(0)\downarrow\}$, $\{e : \varphi_e$ totale$\}$, $\{e : \varphi_e = \varphi_{e_0}\}$, $\{e : \varphi_e(n)\downarrow \Rightarrow \varphi_e(n) = 0\}$

ii. Ne s'applique pas aux propriétés du **programme** («e a moins de 100 états», «s'arrête en $\leq n$ étapes»: décidables)

iii. Preuve (Kleene): $f \in \mathcal{F}$, $g \notin \mathcal{F}$; $h(e, x) = g(x)$ si $e \in \Phi^{-1}(\mathcal{F})$, $f(x)$ sinon; point fixe $\varphi_e = h(e, \cdot)$: contradiction

iv. Preuve (réduction): o.p.s. $\uparrow \notin \mathcal{F}$, $f = \varphi_a \in \mathcal{F}$; $b(e, x)$: «simuler $\varphi_e(x)$, puis renvoyer $\varphi_a(m)$»; $\varphi_{b(e,x)} \in \mathcal{F} \iff (e, x) \in H$

Réductions

Many-to-one: $A \leq_m B$ s'il existe ρ totale calculable avec $m \in A \iff \rho(m) \in B$.

i. $A \leq_m B$, B décidable (resp. semi-déc.) $\Rightarrow A$ aussi

ii. $H \leq_m D \Rightarrow D$ indécidable (méthode standard)

Turing: $A \leq_T B$ si A est décidable par un algorithme pouvant interroger librement (un nombre fini de fois) l'oracle «$n \in B$?».

i. $\leq_m \Rightarrow \leq_T$; $\complement B \leq_T B$ toujours, mais $\complement H \not\leq_m H$

ii. Degrés de Turing: $\mathbf{0}$ (décidables), $\mathbf{0}'$ (degré de H)

Méthode (statut d'un ensemble)

i. **Décidable**: algorithme qui termine toujours (souvent recherche bornée); p.r. si toutes les boucles sont bornées

ii. **Semi-décidable**: recherche non bornée d'un témoin n (arbre de calcul, nombre d'étapes), ou exécution

iii. **Non décidable**: Rice (si propriété de la fonction), réduction de H, ou diagonale par Kleene

iv. **Non semi-décidable**: le complémentaire est semi-décidable mais l'ensemble n'est pas décidable (ex. $\complement H$, $\{e : \varphi_e(0)\uparrow\}$)

λ-calcul: termes

Variable, application (PQ), abstraction $\lambda v.E$.

i. $xyz = ((xy)z)$; $\lambda uv.E = \lambda u.\lambda v.E$; $\lambda x.xy = \lambda x.(xy)$

ii. Variable liée/libre; terme clos: sans variable libre

iii. **α-conversion**: renommer les liées ($\lambda x.x \equiv \lambda y.y$) sans capture ($\lambda y.xy \not\equiv \lambda x.xx$)

β-réduction: un **redex** $(\lambda v.E)T \to E[v \backslash T]$ (substitution sans capture).

i. $(\lambda x.xx)y \to yy$; $(\lambda xy.x)z \to \lambda y.z$; $(\lambda xy.x)y \to \lambda y_1.y$ (renommer!)

ii. $\Omega = (\lambda x.xx)(\lambda x.xx) \to \Omega$

iii. **Forme normale**: aucun redex

i. **Faiblement normalisable**: une suite de réductions termine; **fortement**: toutes

ii. $(\lambda uz.z)\Omega$: faiblement ($\to \lambda z.z$) mais pas fortement normalisable

iii. **Church-Rosser** (confluence): $T \twoheadrightarrow T_1$, $T \twoheadrightarrow T_2 \Rightarrow \exists T''$, $T_1, T_2 \twoheadrightarrow T''$; forme normale unique

iv. **Redex extérieur gauche** (le λ le plus à gauche): stratégie déterministe, normalise tout terme faiblement normalisable

Entiers de Church

$\overline{n} := \lambda fx.f^{\circ n}(x)$: $\overline{0} = \lambda fx.x$, $\overline{1} = \lambda fx.fx$, $\overline{2} = \lambda fx.f(fx)$.

Fonction	Terme
$n + 1$	$\lambda mfx.f(mfx)$
$m + n$	$\lambda mnfx.nf(mfx)$
mn	$\lambda mnf.n(mf)$
m^n	$\lambda mn.nm$
$p = 0\ ?\ m : n$	$\lambda mnp.p(\lambda y.n)m$
couple, proj.	$\lambda rsa.ars$, $\lambda p.p(\lambda rs.r)$

f représentée par t: $t\,\overline{x_1} \cdots \overline{x_k} \twoheadrightarrow_{\text{lft}} \overline{y}$ si $f(x)\downarrow = y$, ne termine pas si $f(x)\uparrow$.

Exemple:
$(\lambda mfx.f(mfx))\overline{n} \to \lambda fx.f(\overline{n}fx) \twoheadrightarrow \lambda fx.f(f^{\circ n}x) = \overline{n+1}$.

Combinateur Y

$$Y := \lambda f.(\lambda x.f(xx))(\lambda x.f(xx))$$
$$Yg \twoheadrightarrow g(Yg) \text{ (point fixe, non normalisable)}$$

i. `let rec h = E`: $Y(\lambda h.E)$ se comporte comme $E[h \backslash Y(\lambda h.E)]$

ii. Implémente μ: $Y(\lambda hz\vec{x}.T(vz\vec{x})z(h(Az)\vec{x}))\,\overline{0}$

iii. Donc: λ-calcul $\equiv$ fonctions récursives

Méthode (λ-calcul)

i. Parenthéser explicitement avant de réduire

ii. Réduire toujours le redex extérieur gauche, renommer en cas de conflit

3. Typage et Curry-Howard

Idée: affecter un type aux termes pour contraindre leur usage. Un système de typage décidable qui garantit la terminaison ne peut pas exprimer son propre interpréteur, ni avoir $t \cong (t \to t)$: aucun terme de type vide.

λ-calcul simplement typé ($\lambda_\to$)

Types: variables $\alpha, \beta, \ldots$ et $\sigma \to \tau$ ($\rho \to \sigma \to \tau = \rho \to (\sigma \to \tau)$). Termes annotés (Church): $\lambda(v : \sigma).E$.

Contexte $\Gamma = x_1 : \sigma_1, \ldots, x_k : \sigma_k$; jugement $\Gamma \vdash E : \tau$.

$$\frac{}{\Gamma, x : \sigma \vdash x : \sigma}\ \text{Var} \qquad \frac{\Gamma \vdash P : \sigma \to \tau \quad \Gamma \vdash Q : \sigma}{\Gamma \vdash PQ : \tau}\ \text{App}$$

$$\frac{\Gamma, v : \sigma \vdash E : \tau}{\Gamma \vdash \lambda(v : \sigma).E : \sigma \to \tau}\ \text{Abs}$$

i. Le terme détermine l'arbre de dérivation: vérification et calcul du type **faciles** (type unique)

ii. Inverse aussi facile: arbre de dérivation (types seuls) $\to$ terme

iii. Difficile: typer un terme non annoté (Hindley-Milner, hors programme); **habitation** d'un type (= prouvabilité)

iv. Affaiblissement, renommage; substitution: $\Gamma, v : \sigma \vdash E : \tau$ et $\Gamma \vdash T : \sigma \Rightarrow \Gamma \vdash E[v \backslash T] : \tau$

v. **Subject reduction**: la β-réduction préserve le type

vi. **Normalisation forte** (Turing, Tait, Girard): toute suite de β-réductions d'un terme typé termine (aussi avec $\times, +, 1, 0$; admis)

vii. Polymorphisme implicite: substituer un type à une variable de type donne encore un terme correct

Exemple: $\vdash \lambda(f : \beta \to \alpha \to \gamma).\lambda(x : \alpha).\lambda(y : \beta).fyx : (\beta \to \alpha \to \gamma) \to \alpha \to \beta \to \gamma$

i. Contexte $\Gamma = f, x, y$; $\Gamma \vdash fy : \alpha \to \gamma$ (App), $\Gamma \vdash fyx : \gamma$ (App), puis 3 Abs

Types enrichis

$$\frac{\Gamma \vdash t_1 : \tau_1 \quad \Gamma \vdash t_2 : \tau_2}{\Gamma \vdash \langle t_1, t_2 \rangle : \tau_1 \times \tau_2} \qquad \frac{\Gamma \vdash t : \tau_1 \times \tau_2}{\Gamma \vdash \pi_i t : \tau_i}$$

$$\frac{\Gamma \vdash t : \tau_i}{\Gamma \vdash \iota_i^{(\tau_1, \tau_2)} t : \tau_1 + \tau_2} \qquad \frac{}{\Gamma \vdash \bullet : 1} \qquad \frac{\Gamma \vdash r : 0}{\Gamma \vdash \text{exfalso}^{(\tau)} r : \tau}$$

$$\frac{\Gamma \vdash r : \sigma_1 + \sigma_2 \quad \Gamma, v_i : \sigma_i \vdash t_i : \tau}{\Gamma \vdash (\text{match } r \text{ with } \iota_1 v_1 \mapsto t_1, \iota_2 v_2 \mapsto t_2) : \tau}$$

Réductions (élimination des **détours** Intro+Élim):

$$(\lambda v.e)t \rightsquigarrow e[v \backslash t], \quad \pi_i \langle t_1, t_2 \rangle \rightsquigarrow t_i$$
$$\text{match } \iota_i s \text{ with} \cdots \rightsquigarrow t_i[v_i \backslash s]$$

Correspondance de Curry-Howard

Programmes	Preuves
type	proposition
terme de type τ	preuve de τ
$\sigma \to \tau$	$P \Rightarrow Q$
$\sigma \times \tau$	$P \wedge Q$
$\sigma + \tau$	$P \vee Q$
1 / 0	$\top$ / $\bot$
$\sigma \to 0$	$\neg P$
variable du contexte	hypothèse
abstraction λ	$\Rightarrow$Int (hyp. déchargée)
application	$\Rightarrow$Élim (modus ponens)
$\langle , \rangle$ / π_i	$\wedge$Int / $\wedge$Élim
ι_i / match	$\vee$Int / $\vee$Élim
exfalso	$\bot$Élim
β-réduction	élimination des détours
normalisation forte	cohérence ($\nvdash \bot$)
$\prod_v$ / $\sum_v$	$\forall$ / $\exists$

i. Exige la terminaison: pas d'appels récursifs (preuves circulaires, paradoxe de Curry)

ii. Logique **intuitionniste**: pas de terme de type $((\alpha \to 0) \to 0) \to \alpha$ ni $\alpha + (\alpha \to 0)$

iii. Preuves non uniques: $\overline{0}_\alpha, \overline{1}_\alpha, \overline{2}_\alpha$ prouvent tous $(A \Rightarrow A) \Rightarrow A \Rightarrow A$

iv. Négation $\sigma \to 0$: «témoignage de vacuité» de σ

Catalogue de λ-termes de preuve

Formule	Terme
$A \Rightarrow A$	$\lambda u.u$
$A \Rightarrow B \Rightarrow A$ (K)	$\lambda uv.u$
S: $(A \Rightarrow B \Rightarrow C) \Rightarrow (A \Rightarrow B) \Rightarrow A \Rightarrow C$	$\lambda xyz.xz(yz)$
$(A \Rightarrow B) \Rightarrow (B \Rightarrow C) \Rightarrow A \Rightarrow C$	$\lambda fgx.g(fx)$
$(B \Rightarrow A \Rightarrow C) \Rightarrow A \Rightarrow B \Rightarrow C$	$\lambda fxy.fyx$
$A \Rightarrow (A \Rightarrow B) \Rightarrow B$	$\lambda uf.fu$
$A \wedge B \Rightarrow B \wedge A$	$\lambda u.\langle \pi_2 u, \pi_1 u \rangle$
$A \Rightarrow A \wedge A$	$\lambda u.\langle u, u \rangle$
$(A \wedge B \Rightarrow C) \Rightarrow A \Rightarrow B \Rightarrow C$	$\lambda fxy.f\langle x, y \rangle$
$(A \Rightarrow B \Rightarrow C) \Rightarrow A \wedge B \Rightarrow C$	$\lambda fz.f(\pi_1 z)(\pi_2 z)$
$A \Rightarrow B \Rightarrow A \wedge B$	$\lambda uv.\langle u, v \rangle$
$A \Rightarrow \neg\neg A$	$\lambda uh.hu$
$(A \Rightarrow B) \Rightarrow \neg B \Rightarrow \neg A$	$\lambda fhu.h(fu)$
$\neg\neg\neg A \Rightarrow \neg A$	$\lambda hu.h(\lambda k.ku)$
$\neg(A \wedge \neg A)$	$\lambda p.(\pi_2 p)(\pi_1 p)$
$\bot \Rightarrow C$	$\lambda r.\text{exfalso}\, r$

Avec $\vee$:

i. $A \vee B \Rightarrow B \vee A$: $\lambda u.\text{match } u \text{ with } \iota_1 v \mapsto \iota_2 v,\ \iota_2 v \mapsto \iota_1 v$

ii. $(A \vee A) \Rightarrow A$: $\lambda u.\text{match } u \text{ with } \iota_1 v \mapsto v,\ \iota_2 v \mapsto v$

iii. $(A \Rightarrow C) \Rightarrow (B \Rightarrow C) \Rightarrow A \vee B \Rightarrow C$: $\lambda fgu.\text{match } u \text{ with } \iota_1 a \mapsto fa,\ \iota_2 b \mapsto gb$

iv. $\neg(A \vee B) \Rightarrow \neg A$: $\lambda hu.h(\iota_1 u)$; $\neg(A \vee B) \Rightarrow \neg A \wedge \neg B$: $\lambda h.\langle \lambda a.h(\iota_1 a), \lambda b.h(\iota_2 b) \rangle$

v. $\neg A \wedge \neg B \Rightarrow \neg(A \vee B)$: $\lambda pu.\text{match } u \text{ with } \iota_1 a \mapsto \pi_1 p\, a,\ \iota_2 b \mapsto \pi_2 p\, b$

vi. $\neg\neg(A \vee \neg A)$: $\lambda h.h(\iota_2(\lambda a.h(\iota_1 a)))$

vii. $(A \vee \neg A) \Rightarrow \neg\neg A \Rightarrow A$: $\lambda zu.\text{match } z \text{ with } \iota_1 v \mapsto v,\ \iota_2 w \mapsto \text{exfalso}(uw)$

Méthode (terme $\leftrightarrow$ formule)

i. Terme $\to$ formule: type de chaque variable liée = hypothèse; type du corps = conclusion; `fun (H:A) (H2:B) => e` prouve $A \Rightarrow B \Rightarrow$(type de e)

ii. Formule $\to$ terme: un λ par hypothèse ($\Rightarrow$), puis appliquer les hypothèses pour atteindre la conclusion; $\neg A$: c'est $A \Rightarrow \bot$, donc encore un λ

4. Calcul propositionnel intuitionniste

Formules: variables $A, B, \ldots, \Rightarrow, \wedge, \vee, \top, \bot$;
$\neg P := P \Rightarrow \bot$.

i. Priorités: $\neg > \wedge > \vee > \Rightarrow$;
$P \Rightarrow Q \Rightarrow R = P \Rightarrow (Q \Rightarrow R)$

ii. Séquent $P_1, \ldots, P_r \vdash Q$: «sous les hypothèses P_i, on a Q»; théorème: $\vdash P$

Déduction naturelle

Axiome: $\Gamma, Q \vdash Q$. Hypothèses **déchargées** entre crochets.

$$\frac{\Gamma, P \vdash Q}{\Gamma \vdash P \Rightarrow Q}\Rightarrow\text{I} \qquad \frac{\Gamma \vdash P \Rightarrow Q \quad \Gamma \vdash P}{\Gamma \vdash Q}\Rightarrow\text{E}$$

$$\frac{\Gamma \vdash Q_1 \quad \Gamma \vdash Q_2}{\Gamma \vdash Q_1 \wedge Q_2}\wedge\text{I} \qquad \frac{\Gamma \vdash Q_1 \wedge Q_2}{\Gamma \vdash Q_i}\wedge\text{E}_i$$

$$\frac{\Gamma \vdash Q_i}{\Gamma \vdash Q_1 \vee Q_2}\vee\text{I}_i \qquad \frac{\Gamma \vdash P_1 \vee P_2 \quad \Gamma, P_1 \vdash Q \quad \Gamma, P_2 \vdash Q}{\Gamma \vdash Q}\vee\text{E}$$

$$\frac{}{\Gamma \vdash \top}\top\text{I} \qquad \frac{\Gamma \vdash \bot}{\Gamma \vdash Q}\bot\text{E} \qquad \frac{\Gamma, \neg Q \vdash \bot}{\Gamma \vdash Q}\text{Abs (classique)}$$

i. Pas d'élimination de $\top$, pas d'introduction de $\bot$

ii. $\neg$Int $= \Rightarrow$I: supposer A, déduire $\bot$, conclure $\neg A$ (valable)

iii. $\neg$Élim $= \Rightarrow$E: de $\neg A$ et A tirer $\bot$

iv. Raisonnement **par l'absurde** (supposer $\neg A$, déduire $\bot$, conclure A): **classique** seulement; intuitionnistement on obtient $\neg\neg A$

Arbre de preuve

$$\dfrac{\dfrac{\dfrac{A \wedge B \vdash A \wedge B}{A \wedge B \vdash B}\wedge\text{E}_2 \qquad \dfrac{A \wedge B \vdash A \wedge B}{A \wedge B \vdash A}\wedge\text{E}_1}{A \wedge B \vdash B \wedge A}\wedge\text{I}}{\vdash A \wedge B \Rightarrow B \wedge A}\Rightarrow\text{I}$$

Présentation drapeau

Numéroter les lignes, ouvrir une boîte par hypothèse, justifier chaque ligne. Ex. $(A \Rightarrow B) \Rightarrow \neg B \Rightarrow \neg A$:

(1)	$\boxed{A \Rightarrow B}$	hyp.
(2)	$\quad\boxed{\neg B}$	hyp.
(3)	$\quad\boxed{A}$	hyp.
(4)	$\quad B$	$\Rightarrow$E (1),(3)
(5)	$\quad \bot$	$\Rightarrow$E (2),(4)
(6)	$\quad \neg A$	$\Rightarrow$I de (3) dans (5)
(7)	$\neg B \Rightarrow \neg A$	$\Rightarrow$I de (2) dans (6)
(8)	$(A \Rightarrow B) \Rightarrow \neg B \Rightarrow \neg A$	$\Rightarrow$I de (1) dans (7)

Terme: $\lambda(f : A \Rightarrow B).\lambda(h : \neg B).\lambda(u : A).h(fu)$.

$\vee$E en drapeau: «$\vee$Élim sur (1) de (2) dans (3) et de (4) dans (5)», deux boîtes ouvertes par les deux cas.

Interprétation BHK

Un témoignage de:

i. $P \wedge Q$: un de P et un de Q

ii. $P \vee Q$: un de P ou un de Q, **et lequel**

iii. $P \Rightarrow Q$: un moyen de transformer un témoignage de P en un de Q

iv. $\top$: trivial; $\bot$: n'existe pas

v. $\forall x.P(x)$: transforme tout x en témoignage de $P(x)$

vi. $\exists x.P(x)$: un x_0 et un témoignage de $P(x_0)$

Intuitionniste vs classique

i. Classique = intuitionniste + tiers exclu (ou absurde, ou $\neg\neg A \Rightarrow A$, ou Peirce)

ii. Intuitionniste: preuves constructives, extraction d'algorithme, propriété de la disjonction

iii. **Propriété de la disjonction**: $\vdash Q_1 \vee Q_2 \Rightarrow \vdash Q_1$ ou $\vdash Q_2$; donc $\nvdash A \vee \neg A$

iv. Le calcul propositionnel intuitionniste est **décidable** (propriété de la sous-formule)

v. Glivenko: P prouvable classiquement ssi $\neg\neg P$ prouvable intuitionnistement

Théorèmes intuitionnistes

i. $(A \Rightarrow B \Rightarrow C) \Leftrightarrow (A \wedge B \Rightarrow C)$

ii. $(A \Rightarrow B) \wedge (A \Rightarrow C) \Leftrightarrow (A \Rightarrow B \wedge C)$

iii. $(A \Rightarrow C) \wedge (B \Rightarrow C) \Leftrightarrow (A \vee B \Rightarrow C)$

iv. $A \wedge (B \vee C) \Leftrightarrow (A \wedge B) \vee (A \wedge C)$; idem $\vee$ sur $\wedge$

v. $\neg A \wedge \neg B \Leftrightarrow \neg(A \vee B)$

vi. $\neg A \vee \neg B \Rightarrow \neg(A \wedge B)$ (seul ce sens)

vii. $A \Rightarrow \neg\neg A$; $\neg\neg\neg A \Leftrightarrow \neg A$

viii. $(A \Rightarrow B) \Rightarrow (\neg B \Rightarrow \neg A)$

ix. $\neg A \vee B \Rightarrow (A \Rightarrow B)$ (seul ce sens)

x. $(\neg\neg A \wedge \neg\neg B) \Leftrightarrow \neg\neg(A \wedge B)$

xi. $(\neg\neg A \Rightarrow \neg\neg B) \Leftrightarrow \neg\neg(A \Rightarrow B)$

xii. $\neg(A \wedge \neg A)$; $\neg\neg(A \vee \neg A)$

xiii. $(A \vee \neg A) \Rightarrow (\neg\neg A \Rightarrow A)$

Non-théorèmes intuitionnistes

Valables classiquement, pas intuitionnistement:

i. $A \vee \neg A$ (tiers exclu)

ii. $\neg\neg A \Rightarrow A$ (double négation)

iii. $((A \Rightarrow B) \Rightarrow A) \Rightarrow A$ (Peirce)

iv. $\neg(A \wedge B) \Rightarrow \neg A \vee \neg B$ (De Morgan)

v. $(A \Rightarrow B) \Rightarrow \neg A \vee B$

vi. $(\neg B \Rightarrow \neg A) \Rightarrow (A \Rightarrow B)$

vii. $(A \Rightarrow B) \vee (B \Rightarrow A)$ (Dummett)

viii. $\neg A \vee \neg\neg A$ (tiers exclu faible)

ix. $(\neg\neg A \Rightarrow A) \Rightarrow A \vee \neg A$

x. $((\neg\neg A \Rightarrow A) \Rightarrow A \vee \neg A) \Rightarrow \neg\neg A \vee \neg A$ (Scott)

xi. $((A \Rightarrow B) \Rightarrow B) \wedge ((B \Rightarrow A) \Rightarrow A) \Rightarrow A \vee B$

Tester d'abord la table de vérité.

Méthode (prouver une formule)

i. Introduire les hypothèses ($\Rightarrow$I)

ii. But $\wedge$: deux sous-preuves; but $\vee$: choisir le côté (souvent après un $\vee$E)

iii. Hyp. $\vee$: $\vee$E; but $\neg A$: supposer A, viser $\bot$

5. Sémantiques

Sémantique: ensemble de modèles où chaque formule a une valeur.

i. **Correcte**: tout théorème est valide dans tout modèle
ii. **Complète**: toute formule valide partout est un théorème
iii. **Usage**: trouver un modèle où φ n'est pas valide $\Rightarrow$ par correction, φ n'est pas démontrable

Sémantique	Correcte	Complète
Booléenne	oui	classique
Kripke	oui	oui (tous cadres)
Ouverts	oui	oui (même $\mathbb{R}^n$)
Réalisabilité	oui	non
Pb. finis (Medvedev)	oui	non

Sémantique des ouverts

X espace topologique (en pratique $\mathbb{R}$), valeurs = ouverts:

$$\dot\top = X, \quad \dot\bot = \emptyset, \quad U\dot\wedge V = U\cap V, \quad U\dot\vee V = U\cup V$$
$$U\dot\Rightarrow V = \operatorname{int}\big((X\setminus U)\cup V\big)$$
$$\dot\neg U = \operatorname{int}(X\setminus U), \quad \dot\neg\dot\neg U = \operatorname{int}(\overline{U})$$

i. $U\dot\Rightarrow V$: plus grand ouvert W avec $U\cap W\subseteq V$; $= X$ ssi $U\subseteq V$
ii. $\dot\neg U$: plus grand ouvert disjoint de U
iii. φ valide ssi $\dot\varphi = X$ pour tous ouverts (Tarski)
iv. Toujours vérifier que chaque valeur est **ouverte**; attention aux points frontière (souvent 0)

Valeurs utiles sur $\mathbb{R}$:

i. $U=]0,+\infty[$: $\dot\neg U=]-\infty,0[$, $\dot\neg\dot\neg U = U$, $U\dot\vee\dot\neg U = \mathbb{R}\setminus\{0\}$
ii. $U=\mathbb{R}\setminus\{0\}$: $\dot\neg U=\emptyset$, $\dot\neg\dot\neg U=\mathbb{R}$, $\dot\neg\dot\neg U\dot\Rightarrow U = U\neq\mathbb{R}$
iii. $U=]0,1[$: $\dot\neg U=]-\infty,0[\cup]1,+\infty[$, $U\dot\vee\dot\neg U = \mathbb{R}\setminus\{0,1\}$
iv. $U=]-\infty,0[$, $V=]0,+\infty[$: $U\dot\Rightarrow V = V$, $V\dot\Rightarrow U = U$, $U\cap V=\emptyset$

Sémantique de Kripke

Cadre: ensemble ordonné $(W,\leq)$ de **mondes**. Affectation $p: W\to\{0,1\}$ **croissante** (permanente: vrai en $w\Rightarrow$ vrai en tout $w'\geq w$).

i. $(q_1\dot\wedge q_2)(w)=1$ ssi les deux valent 1 en w; $\dot\vee$: l'un des deux
ii. $(q_1\dot\Rightarrow q_2)(w)=1$ ssi $\forall w'\geq w$, $q_1(w')=1\Rightarrow q_2(w')=1$
iii. $\dot\neg p(w)=1$ ssi $p=0$ dans tout $w'\geq w$
iv. $\dot\neg\dot\neg p(w)=1$ ssi $\forall w'\geq w$, $\exists w''\geq w'$, $p(w'')=1$
v. φ validée par le modèle: vaut 1 dans **tout** monde
vi. Correcte et complète; singleton = booléenne

Cadre $u\leq v$, p vrai seulement en v:

i. $\dot\neg p=0$ partout, $\dot\neg\dot\neg p=1$ partout
ii. $p\dot\vee\dot\neg p$: 0 en $u\Rightarrow$ tiers exclu non prouvable
iii. $\dot\neg\dot\neg p\dot\Rightarrow p$: 0 en $u\Rightarrow$ double négation
iv. $q\equiv 0$: $p\dot\Rightarrow q=0$ partout, $(p\dot\Rightarrow q)\dot\Rightarrow p=1$, Peirce vaut 0 en u

Cadre $u\leq v$**,** $u\leq w$ (v, w incomparables), p vrai seulement en v, q seulement en w:

i. $p\dot\Rightarrow q$, $q\dot\Rightarrow p$: 0 en $u\Rightarrow$ Dummett vaut 0 en u
ii. $\dot\neg(p\dot\wedge q)=1$ partout, $\dot\neg p\dot\vee\dot\neg q=0$ en $u\Rightarrow$ De Morgan
iii. $\dot\neg p\dot\vee\dot\neg\dot\neg p=0$ en $u\Rightarrow$ tiers exclu faible
iv. $(p\dot\Rightarrow q)\dot\Rightarrow q=1$ partout (et symétrique), $p\dot\vee q=0$ en $u\Rightarrow$ CC 24-25

Réalisabilité propositionnelle

Valeurs = parties de $\mathbb{N}$, Φ_e = e-ième fonction récursive:

i. $P\dot\wedge Q=\{\langle m,n\rangle : m\in P, n\in Q\}$; $P\dot\vee Q=\{\langle 0,m\rangle : m\in P\}\cup\{\langle 1,n\rangle : n\in Q\}$
ii. $P\dot\Rightarrow Q=\{e : \forall m\in P.\,\Phi_e(m)\downarrow\in Q\}$; $\dot\top=\mathbb{N}$, $\dot\bot=\emptyset$
iii. $\dot\neg P=\mathbb{N}$ si $P=\emptyset$, $\emptyset$ sinon
iv. φ réalisable: un **même** n réalise φ pour toutes parties P_i
v. Correcte: réalisateur extrait du λ-terme
vi. $A\vee\neg A$, $\neg\neg A\Rightarrow A$: non réalisables

Exemple complet: formule de Scott (CC 25-26)

$((\neg\neg A\Rightarrow A)\Rightarrow A\vee\neg A)\Rightarrow\neg\neg A\vee\neg A$ sur $\mathbb{R}$ avec

$$U=]0,+\infty[\setminus\{1,\tfrac12,\tfrac14,\dots\}=]1,\infty[\cup]\tfrac12,1[\cup]\tfrac14,\tfrac12[\cup\cdots$$

(ouvert: réunion d'intervalles ouverts)

i. $\dot\neg U=]-\infty,0[$, $\dot\neg\dot\neg U=]0,+\infty[$
ii. $\dot\neg\dot\neg U\dot\Rightarrow U = U\cup]-\infty,0[$ (0 exclu: tout voisinage de 0 contient des 2^{-n})
iii. $U\dot\vee\dot\neg U = U\cup]-\infty,0[$: même ouvert
iv. Donc $(\dots)\dot\Rightarrow(\dots)=\mathbb{R}$
v. $\dot\neg\dot\neg U\dot\vee\dot\neg U=\mathbb{R}\setminus\{0\}$; formule entière: $\mathbb{R}\setminus\{0\}\neq\mathbb{R}$: non démontrable

Exemple: CC 24-25

$((A\Rightarrow B)\Rightarrow B)\wedge((B\Rightarrow A)\Rightarrow A)\Rightarrow A\vee B$, $U=]-\infty,0[$, $V=]0,\infty[$:

i. $U\dot\Rightarrow V=V$, donc $(U\dot\Rightarrow V)\dot\Rightarrow V=\mathbb{R}$; symétriquement $\mathbb{R}$
ii. $U\dot\vee V=\mathbb{R}\setminus\{0\}$: formule $=\mathbb{R}\setminus\{0\}$, non démontrable
iii. Classiquement prouvable (table de vérité: si A,B faux, $(A\Rightarrow B)\Rightarrow B$ est faux)

Méthode (montrer qu'une formule n'est pas démontrable)

i. Table de vérité: si faux pour une affectation, non prouvable même classiquement
ii. Sinon, **ouverts de** $\mathbb{R}$: $]0,\infty[$, $\mathbb{R}\setminus\{0\}$, ou $]-\infty,0[$ et $]0,\infty[$
iii. Ou **Kripke**: cadre $u\leq v$ (une variable) ou $u\leq v,w$ (deux), variables vraies seulement aux mondes maximaux
iv. Calculer chaque sous-formule (vérifier ouverture/croissance)
v. Conclure: «la valeur n'est pas X tout entier (resp. vaut 0 en u), donc par correction de la sémantique, la formule n'est pas démontrable»

6. Quantificateurs et arithmétique

$\forall, \exists = \wedge, \vee$ «en famille»: Curry-Howard $\prod_v \sigma(v)$ (fonction dépendante) et $\sum_v \sigma(v)$ (couple dépendant).

Règles

$$\frac{\Gamma, v : I \vdash Q}{\Gamma \vdash \forall v.Q}\,\forall\text{I} \qquad \frac{\Gamma \vdash \forall v.Q \quad \Gamma \vdash t : I}{\Gamma \vdash Q[v\backslash t]}\,\forall\text{E}$$

$$\frac{\Gamma \vdash t : I \quad \Gamma \vdash Q[v\backslash t]}{\Gamma \vdash \exists v.Q}\,\exists\text{I} \qquad \frac{\Gamma \vdash \exists v.P \quad \Gamma, v : I, P \vdash Q}{\Gamma \vdash Q}\,\exists\text{E}$$

i. **Frais**: v n'apparaît libre dans aucune hypothèse en cours (ni dans Q pour $\exists$E)

ii. $\forall$I: «soit v arbitraire… donc $\forall v.Q(v)$»; $\forall$E: «en particulier $Q(t)$»

iii. $\exists$I: «pour ce t, $Q(t)$, donc $\exists v.Q$»; $\exists$E: «soit v tel que $P(v)\dots$»

Termes:

i. $\forall$I: $\lambda(v : I).s$; $\forall$E: ft

ii. $\exists$I: $\langle t, z\rangle$; $\exists$E: match z with $\langle v, h\rangle \mapsto s$

Logique du premier ordre

Variables d'individus (type unique I, non écrit), variables de prédicats n-aires $A(x_1, \dots, x_n)$; on ne quantifie que sur les individus.

i. I pas supposé habité: $\nvdash \exists x.\top$, $\nvdash \forall x.A(x) \Rightarrow \exists x.A(x)$; mais $\vdash \forall z.\exists x.\top$ $(\lambda z.\langle z, \bullet\rangle)$

ii. Pour former $t : I$, il faut une variable du contexte

iii. Convention: $\forall$, $\exists$ moins prioritaires que les connecteurs

Égalité: réflexivité $\forall x.x = x$ et schéma de substitution $\forall x \forall y.(x = y \Rightarrow P(x) \Rightarrow P(y))$.

i. Symétrie: $\lambda xyu.\mathrm{subst}_{(\lambda s.s=x)}\, x\, y\, u\, (\mathrm{refl}\, x)$

ii. Transitivité: $\lambda xyzuv.\mathrm{subst}_{(\lambda s.x=s)}\, y\, z\, v\, u$

Théorèmes du premier ordre

i. $(\exists x.\forall y.B(x,y)) \Rightarrow \forall y'.\exists x'.B(x', y')$: $\lambda u.\lambda y'.$match u with $\langle x, v\rangle \mapsto \langle x, vy'\rangle$ (réciproque fausse)

ii. $(\forall x.(A(x) \Rightarrow C)) \Rightarrow (\exists x.A(x)) \Rightarrow C$: $\lambda fp.$match p with $\langle x, w\rangle \mapsto fxw$

iii. $((\exists x.A(x)) \Rightarrow C) \Rightarrow \forall x.(A(x) \Rightarrow C)$: $\lambda gxu.g\langle x, u\rangle$

iv. D'où $(\forall x.\neg A(x)) \Leftrightarrow \neg\exists x.A(x)$

v. $(\exists x.\neg A(x)) \Rightarrow \neg\forall x.A(x)$: $\lambda pf.$match p with $\langle x, w\rangle \mapsto w(fx)$ (réciproque classique seulement)

vi. $(\forall x.A(x)) \Rightarrow (\exists x.\top) \Rightarrow \exists x.A(x)$: $\lambda fp.$match p with $\langle x, u\rangle \mapsto \langle x, fx\rangle$

vii. $(\forall x.P(x)) \vee (\forall x.Q(x)) \Rightarrow \forall x.(P(x) \vee Q(x))$ (CC 25-26): $\lambda h.\lambda x.$match h with $\iota_1 g \mapsto \iota_1(gx)$, $\iota_2 g \mapsto \iota_2(gx)$

viii. $(C \wedge \forall x.B(x)) \Rightarrow \forall x.(C \wedge B(x))$: $\lambda p.\lambda x.\langle \pi_1 p, (\pi_2 p)x\rangle$

ix. Classique seulement: $\neg\forall x.A(x) \Rightarrow \exists x.\neg A(x)$

Arithmétique de Heyting / Peano

HA (intuitionniste) et PA (classique): mêmes axiomes. Termes: 0, Sn, $+$, $\times$, $\triangle$ (puissance).

i. $\forall n.\neg(Sn = 0)$; $\forall mn.(Sm = Sn \Rightarrow m = n)$

ii. **Récurrence** (schéma): $P(0) \Rightarrow (\forall n.(P(n) \Rightarrow P(Sn))) \Rightarrow \forall n.P(n)$

iii. $m + 0 = m$, $m + Sn = S(m + n)$; $m \times 0 = 0$, $m \times Sn = m \times n + m$; $m \triangle 0 = S0$, $m \triangle Sn = (m \triangle n) \times m$

iv. Curry-Howard: la récurrence est une **récursion primitive** dépendante

v. Ex.: $\forall n.(n = 0 \vee \neg n = 0)$ prouvable dans HA par récurrence

Propriétés de HA

i. Propriété de la **disjonction** et de l'**existence** (témoin explicite)

ii. **Extraction**: HA $\vdash \forall m.\exists n.Q(m,n) \Rightarrow$ il existe φ_e totale avec $\forall m.Q(m, \varphi_e(m))$

iii. $\varphi_e(i)\!\downarrow$ formalisable: $\exists n.T(n, e, i)$

iv. $\forall e \forall i.(\varphi_e(i)\!\downarrow \vee \neg\varphi_e(i)\!\downarrow)$: prouvable dans PA, **pas** dans HA (sinon on extrairait un algorithme pour l'arrêt)

v. Traduction double négation: PA $\vdash P \iff$ HA $\vdash P^{\mathrm{CPS}}$

Démontrabilité

i. **Vérifier une preuve**: décidable (même p.r.): prédicat $\mathrm{Pf}(n, k)$

ii. **Être un théorème**: $\exists n.\mathrm{Pf}(n, k)$, seulement **semi-décidable** (parcourir les preuves)

iii. Si $\varphi_e(i)\!\downarrow$, la trace d'exécution donne une preuve de $\varphi_e(i)\!\downarrow$ dans HA; une boucle infinie explicite est aussi prouvable

Théorème de Gödel

Programme g (par Kleene): cherche en parallèle une preuve de «g termine» et de «g ne termine pas»; si la 1re: boucle infinie; si la 2e: termine.

i. Supposons Consis(PA) (PA $\nvdash \bot$)

ii. Preuve de non-terminaison trouvée $\Rightarrow$ g termine $\Rightarrow$ prouvable $\Rightarrow \bot$

iii. Preuve de terminaison trouvée $\Rightarrow$ boucle explicite $\Rightarrow$ prouvable $\Rightarrow \bot$

iv. Donc g ne termine pas, **non prouvable** dans PA; et PA $\nvdash$ Consis(PA)

Généralisation: toute T à preuves vérifiables, formalisant $\varphi_e(i)\!\downarrow$: si Consis(T), T incomplète, $T \nvdash$ Consis(T) (PA, ZFC, Rocq…).

i. L'ensemble des théorèmes est semi-décidable **non décidable** (même argument diagonal)

7. Rocq (Coq)

Idée: assistant de preuve fondé sur Curry-Howard (calcul des constructions inductives): `Prop` pour les propositions, `Type` pour les données; une preuve est un terme, une tactique construit ce terme. Noms des tactiques exigibles.

Logique: tactique ↔ règle

But / hyp.	Tactique	Règle
but $\to$, $\neg$, $\forall$	`intros H`	$\Rightarrow$I,$\forall$I
hyp. $H : A \to B$	`apply H`	$\Rightarrow$E
but $\wedge$, $\leftrightarrow$	`split`	$\wedge$I
hyp. $A \wedge B$	`destruct H as (H1,H2)`	$\wedge$E
but $A \vee B$	`left` / `right`	$\vee$I
hyp. $A \vee B$	`destruct H as [H1|H2]`	$\vee$E
but $\exists x, P\,x$	`exists t`	$\exists$I
hyp. $\exists x, P\,x$	`destruct H as (x,Hx)`	$\exists$E
but $=$ hypothèse	`assumption`	Ax
terme connu	`exact t`	
hyp. `False`	`contradiction`	$\bot$E
but $\to$ `False`	`exfalso`	$\bot$E
lemme	`assert (H:P)`	coupure

i. `~A` est `A -> False`: `intros` sur un but $\neg A$ donne but `False`; `apply H2` avec `H2 : ~B` transforme le but `False` en `B`

ii. `apply H in H'`: raisonnement en avant

iii. Imbriqué: `destruct H as ((H1,H2),H3)`

iv. `contradiction` aussi si H et $\neg H$ sont dans le contexte

Termes de preuve Rocq

i. `fun (H1:A) (H2:B) => H2` : $A \to B \to B$

ii. `fun (H1:A) (H2:~A) => H2 H1` : $A \to \neg\neg A$

iii. `fun H1 H2 H3 => H1 H3 H2` avec `H1:A->B->C`, `H2:B`, `H3:A` : $(A \to B \to C) \to B \to A \to C$

iv. `fun (H1:A->B) (H2:B->C) (H3:A) => H2 (H1 H3)` : $(A \to B) \to (B \to C) \to A \to C$

v. `conj`, `or_introl`/`or_intror`, `ex_intro`, `match`: constructeurs et éliminations

Types inductifs

i. `split`, `left`, `right`: appliquer un constructeur (unique, 1er, 2e)

ii. `destruct x`: analyse de cas sur les constructeurs (récupère les arguments)

iii. `induction n as [|n IHn]`: récurrence (principe engendré par le type); donne `IHn`

iv. `injection H as H'`: de $C\,a = C\,b$ tirer $a = b$ (constructeurs injectifs); ex. `S n = S m` $\to$ `n = m`

v. `discriminate`: hypothèse $C_1 \ldots = C_2 \ldots$ avec constructeurs différents: absurde, prouve tout

vi. `inversion`: analyse de cas sur un prédicat inductif

Calcul et égalité

i. `simpl`: simplifie (calcule) lisiblement; `S n + 0` $\to$ `S (n + 0)`

ii. `unfold f`: déplie une définition; `lazy`: réductions données

iii. `rewrite H` / `rewrite <- H`: remplace le membre gauche (droit) de l'égalité H

iv. `reflexivity`: deux membres identiques à calcul près

v. `symmetry`, `transitivity t`

vi. `f_equal`: `S (n+0) = S n` $\to$ `n+0 = n`

vii. `lia`: arithmétique linéaire automatique

viii. Toutes acceptent `in H` ou `in *`

`rewrite` ou `apply` un lemme?

Lemme `L : forall n, n * 0 = 0`:

i. `rewrite L`: le but doit **contenir un sous-terme** de la forme `?n * 0`; `rewrite <- L`: un sous-terme `0`

ii. `apply L`: le but entier doit être **convertible** (à calcul près) à `?n * 0 = 0`

iii. `n * 0 = 0 + 0`: les deux (`0+0` se calcule en `0`)

iv. `(n+m) * 0 = n*0 + m*0`: `rewrite` seulement

v. Avec `add_0_r : n + 0 = n`: `n + 0 = 0 + n` se réduit à `n + 0 = n`: les deux

Définitions

```
Inductive jour : Type :=
 | lundi | mardi | mercredi | jeudi
 | vendredi | samedi | dimanche.
Definition suivant (j:jour) : jour :=
 match j with lundi => mardi
 | ... | dimanche => lundi end.
Lemma dim : suivant dimanche = lundi.
Proof. simpl. reflexivity. Qed.
Lemma neq : forall j, suivant j <> j.
Proof. intros j H. destruct j;
  simpl in H; discriminate. Qed.

Inductive arbre : Type :=
 | feuille : arbre
 | noeud : nat -> arbre -> arbre -> arbre.
Fixpoint miroir (a:arbre) : arbre :=
 match a with
 | feuille => feuille
 | noeud v g d =>
     noeud v (miroir d) (miroir g)
 end.
Lemma miroir_inv : forall a,
  miroir (miroir a) = a.
Proof. induction a; simpl.
 - reflexivity.
 - rewrite IHa1, IHa2. reflexivity.
Qed.
```

Exemples d'états (CC)

i. `H:(A/\B)/\C` $\to$ `H1:A/\B`, `H2:C`: `destruct H as (H1,H2)`

ii. But `P <-> Q` $\to$ deux buts `P->Q`, `Q->P`: `split`

iii. `H1:B` $\vdash$ `A\/B` $\to$ fini: `right. assumption.`

iv. But `~A` $\to$ `H3:A` $\vdash$ `B` (avec `H2:~B`): `intros H3. apply H2.`

Méthode

i. But: intro du connecteur; hyp.: `destruct`/`apply`; récursif: `induction`